NFCU DataPower OpenShift Deployment Lab Guide

Version: 3.2.6
Last Updated: July 22, 2026
Lab Format: Hands-on

Table of Contents

Contents

Lab Goal

Deploy IBM DataPower Gateway on OpenShift with the web management UI (for demo only).

What You Will Complete

By the end of this lab, you will:

1. Log in to OpenShift

2. Create or switch to the `nfcu-demo` project
3. Install the IBM DataPower Operator
4. Create the required secrets and ConfigMaps for the base deployment
5. Deploy the DataPower service
6. Expose the web UI and MPG route
7. Validate that the deployment is working
8. Pull configuration from a Git repository using the DataPower GitOps feature

---

Lab Inputs You Need Before Starting

You may now access your OpenShift portal here:

OpenShift Console URL:
https://console-openshift-console.apps.<cluster-name>.<domain>

Credentials for OpenShift Portal:

Field	Value
Username	`kubeadmin`
Password	`<openshift-password>`

Infrastructure Node:

Field	Value
API / Infrastructure Node	`api.<cluster-name>.<domain>`
Infrastructure Public IP	`<public-ip>`

Credentials for Infrastructure Node:

Field	Value
Username	`root`

Field	Value
Password	`<root-password>`

Additional Lab Inputs:

Variable	Your Value
IBM entitlement key	
Email for entitlement secret	

How to Get Your OpenShift Login Information

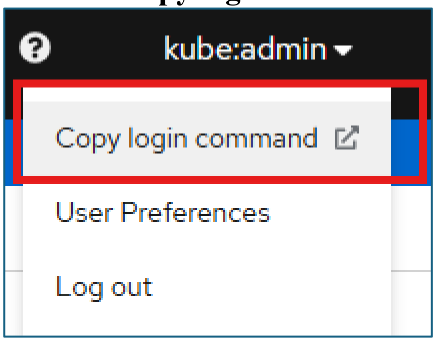
Use one of these methods:

- Open the OpenShift web console and use **Copy login command** from the top-right menu.

- Ask your OpenShift cluster administrator or lab instructor for your cluster credentials.

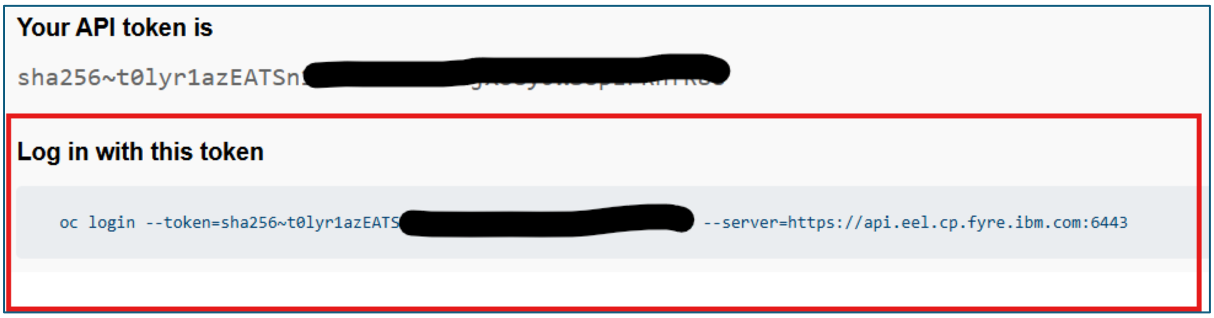
From the OpenShift console:

1. Log in to the OpenShift web console.
2. In the top-right corner, click your user menu.
3. Select **Copy login command**.



4. Choose **Display Token** if prompted.

5. Copy the provided `oc login` command.



Example username/password login:

Command: `oc login https://api.<cluster-name>:6443 -u <username> -p <password>`

Files Used in This Lab

Download the lab files from the following GitHub repository before starting:

[GitHub Repo]()

The following files from that repository are used in this lab:

- `Datapower.yaml`
- `web-mgmt.cfg`
- `webgui-service.yaml`
- `webgui-route.yaml`
- `route.yaml`
- `service.yaml`
- `pull-from-git.cfg`
- `github.com.cer`

Lab Prerequisites

Complete these before the lab:

- Have access to an OpenShift cluster
- Have the `oc` CLI installed
- Obtain your IBM entitlement key
- Know the OpenShift apps domain for your cluster

Lab 1: Log in to OpenShift

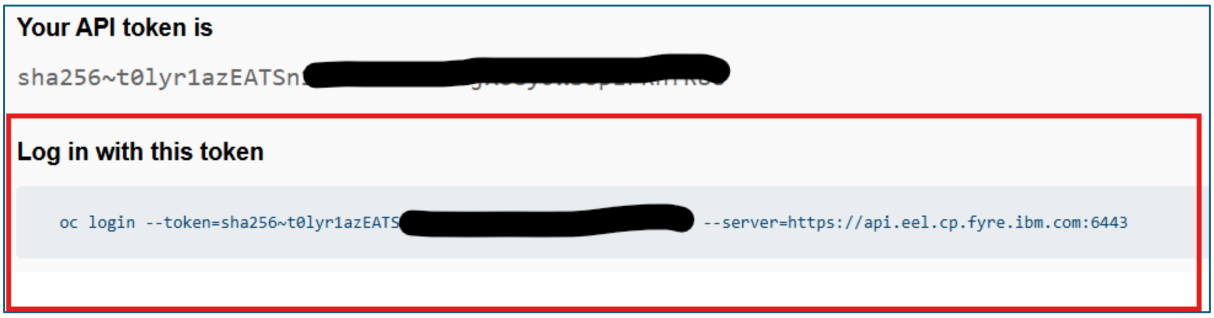
Objective

Authenticate to the cluster and confirm your CLI access works.

Steps

1. Get your login command from the OpenShift console using **Copy login command**, or use the username and password provided by your administrator.
2. Run the token-based login command:

Command: oc login --token=<token> --server=https://api.<cluster-name>:6443



3. Confirm your login:

Command: `oc whoami`

Command: `oc cluster-info`

4. Record the current user and cluster information.

Checkpoint

- `oc whoami` returns your username
- `oc cluster-info` returns cluster details

Lab 2: Create or Select the Project

Objective

Use the target namespace for the operator and DataPower instance.

Steps

1. Create the project if needed:

Command: `oc new-project nfcu-demo`

2. If it already exists, switch to it:

Command: `oc project nfcu-demo`

3. Confirm the current project:

Command: `oc project`

Checkpoint

- Current project is `nfcu-demo`

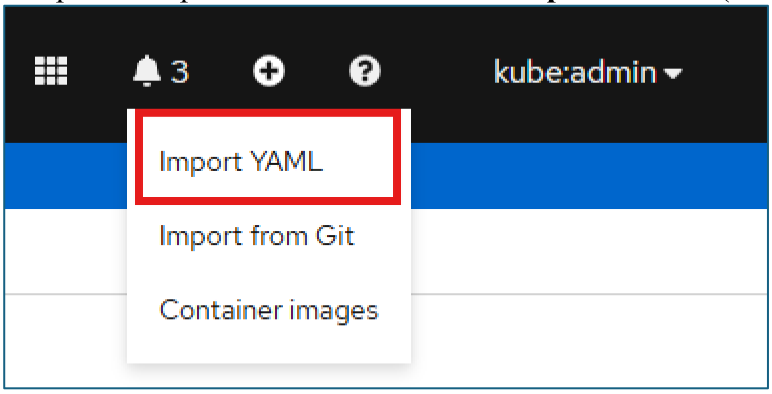
Lab 3: Install the DataPower Operator

Objective

Install the catalog source and operator subscription.

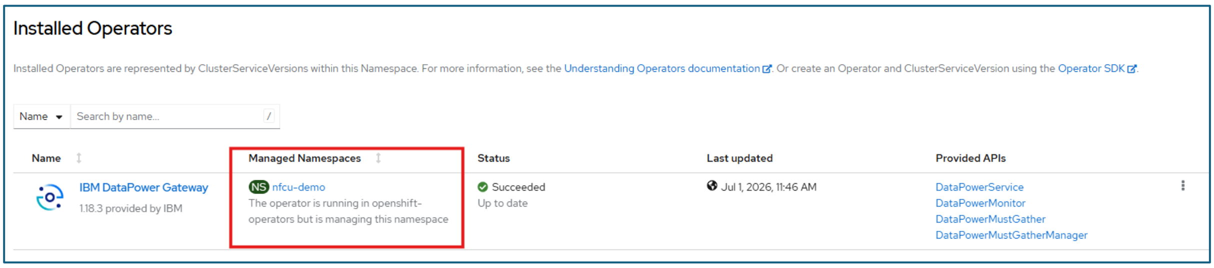
Steps

1. Open the `datapower-catalog-source-latest.yaml` file and copy the contents.
2. Open the OpenShift console and use **Import YAML** (the + icon in the top-right toolbar).



3. Paste the contents of the `datapower-catalog-source-latest.yaml` YAML into the editor.
4. Make sure the namespace is set to `openshift-marketplace for this POT`, then click **Create**.
5. After the resource is created, open the catalog source in the console and select the **YAML** tab to review the applied resource definition.
6. Open the IBM DataPower Operator.
   - Confirm the operator is available from the expected catalog source before continuing.
7. Install the DataPower Operator from the OpenShift console:
   - In **Operators** > **OperatorHub**, search for `DataPower`.
   - Select the **IBM DataPower Gateway** tile.
   - In the panel on the right, click **Install**.
   - Under **Installation Mode**, select your desired installation mode.
   - Select the desired **Update Channel**.
   - Select the desired **Approval Strategy**.
     - If **Automatic** is selected, operator updates in the chosen channel are applied automatically when available.

        - If **Manual** is selected, an administrator must approve each operator update through OLM.
    - Click **Subscribe** to install the DataPower Operator.
    - Make sure Operator Installs Successfully
8. Go to **Operators** > **Installed Operators**.
    - View installed operators in the `openshift-operators` namespace.
    - Confirm **IBM DataPower Gateway** appears in the list with no installation errors.



Checkpoint

- Subscription exists
- ClusterServiceVersion (CSV) for DataPower is installed
- Operator pod is running

Lab 4: Prepare Base Deployment Secrets and ConfigMaps

Objective

Create the resources required for the base DataPower deployment.

Steps

1. Make sure you are in the `nfcu-demo` project:

Command: `oc project nfcu-demo`

2. Create the admin password secret:

Command: `oc create secret generic dp-admin-password --from-literal=password=admin -n nfcu-demo`

3. Create the GitHub certificate secret referenced by `Datapower.yaml`:

Command: `oc create secret generic github.com.cer --from-file=github.com.cer -n nfcu-demo`

4. Create the IBM entitlement secret:

Note: This step allows the DataPower Operator to pull the DataPower image from the IBM Container Registry.

If you do not yet have an entitlement key, retrieve one here: https://myibm.ibm.com/products-services/containerlibrary

Run the following command to create the secret:

Command: `oc create secret docker-registry ibm-entitlement-key --docker-server=cp.icr.io --docker-username=cp --docker-password=<YOUR_IBM_ENTITLEMENT_KEY> --docker-email=<YOUR_EMAIL> -n nfcu-demo`

5. Link the secret to the default service account for image pulls:

Command: `oc secrets link default ibm-entitlement-key --for=pull -n nfcu-demo`

6. Create the web management ConfigMap from the `web-mgmt.cfg` file:

Command: `oc create configmap web-mgmt-cfg --from-file=web-mgmt.cfg -n nfcu-demo`

7. Verify each resource:

Command: `oc get configmap web-mgmt-cfg -n nfcu-demo`

Command: `oc get secret dp-admin-password -n nfcu-demo`

Command: `oc get secret github.com.cer -n nfcu-demo`

Command: `oc get secret ibm-entitlement-key -n nfcu-demo`

Checkpoint

- `web-mgmt-cfg` exists
- `dp-admin-password` exists
- `github.com.cer` exists
- `ibm-entitlement-key` exists
- Entitlement key is linked to the default service account

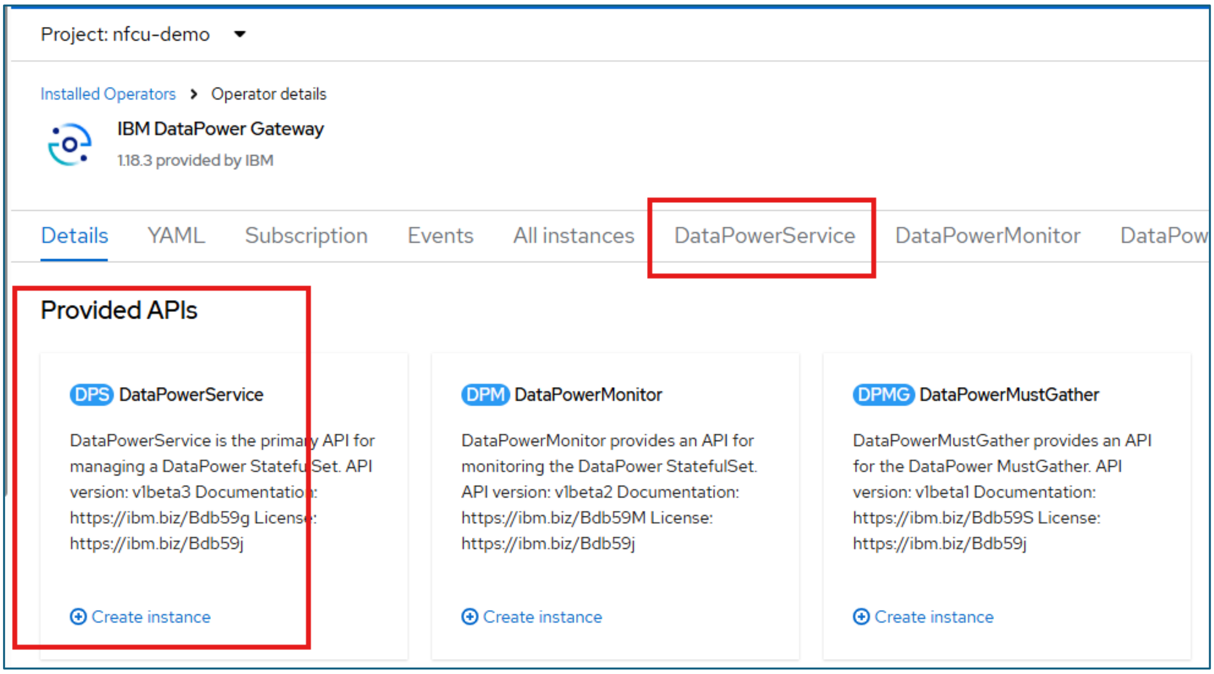
Lab 5: Deploy DataPower

Objective

Create the DataPowerService resource and wait for readiness.

Steps

1. Review `Datapower.yaml` and confirm:
    - namespace is `nfcu-demo`
    - version matches your operator-supported version
    - deployment name is `quickstart`
    - the `default` domain includes the `web-mgmt-cfg` ConfigMap
    - the configured domains are `default` and `demo`
2. In the OpenShift console, go to **Operators** > **Installed Operators**.
3. Open **IBM DataPower Gateway**.
4. Under **Provided APIs**, click **DataPowerService**.



5. Click **Create Instance**.

Note: Make sure Project is nfcu-demo

6. Open the **Form view** first and review what is there by default the main fields that map to `Datapower.yaml`, including:

  - DataPowerService name
  - namespace
  - license acceptance
  - license ID
  - license use
  - version
  - users
  - domains
  - replicas

7. Switch to the **YAML view**, paste the content from `Datapower.yaml`, then click **Create**. Note - wait for Status of "**Running**".

8. Optional: Watch the deployment progress:

Command: `oc get pods -w`

9. Optional: In another terminal, verify service status:

Command: `oc get datapowerservice quickstart`

Command: `oc logs quickstart-0`

Checkpoint

- Pod reaches `Running`
- DataPowerService becomes ready
- Base DataPower startup completes successfully

Lab 6: Expose the Web UI and MPG

Objective

Create the services and routes needed for access.

Steps

1. Open `webgui-route.yaml`
2. You will update `spec.host` before applying it.
3. The `spec.host`, uses this pattern:
   `<route-or-service-name>-<namespace>.<hostname>`

4. See below:

webgui-route.yaml

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: quickstart-webgui
  namespace: nfcu-demo
spec:
  host: quickstart-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com
  port:
    targetPort: webgui-9090
  tls:
    insecureEdgeTerminationPolicy: None
    termination: passthrough
  to:
    kind: Service
    name: quickstart-webgui
    weight: 100
  wildcardPolicy: None

# update host name
```

5. To find your cluster apps domain, run:

   Command: `oc get ingresses.config/cluster -o jsonpath='{.spec.domain}'`

6. Replace <hostname> with output from the above command.
7. Save webgui-route.yaml after updating the host name

2. Apply the updated web UI service:

Command: `oc apply -f webgui-service.yaml`

3. Apply the web UI route to expose port 9090:

Command: `oc apply -f webgui-route.yaml`

4. Apply the MPG service:

Command: `oc apply -f service.yaml`

5. Apply the MPG route to expose port 6000:

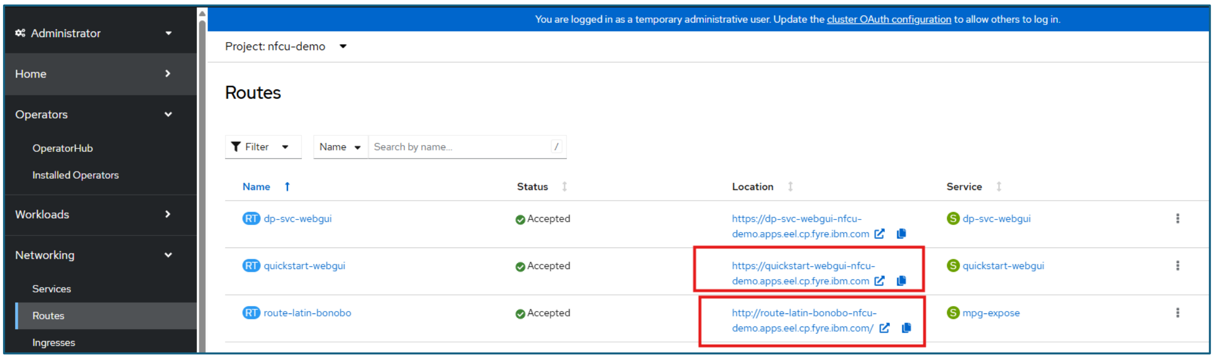
Command: `oc apply -f route.yaml`

6. List all routes:

Command: `oc get route`

7. Validate that the Web UI and MPGW route exists via the Openshift Console as well

from Console: OpenShift Console-->Networking--Routes--Select The quickstart-webgui-->Select Location



Checkpoint

- Web UI route exists
- MPG route exists

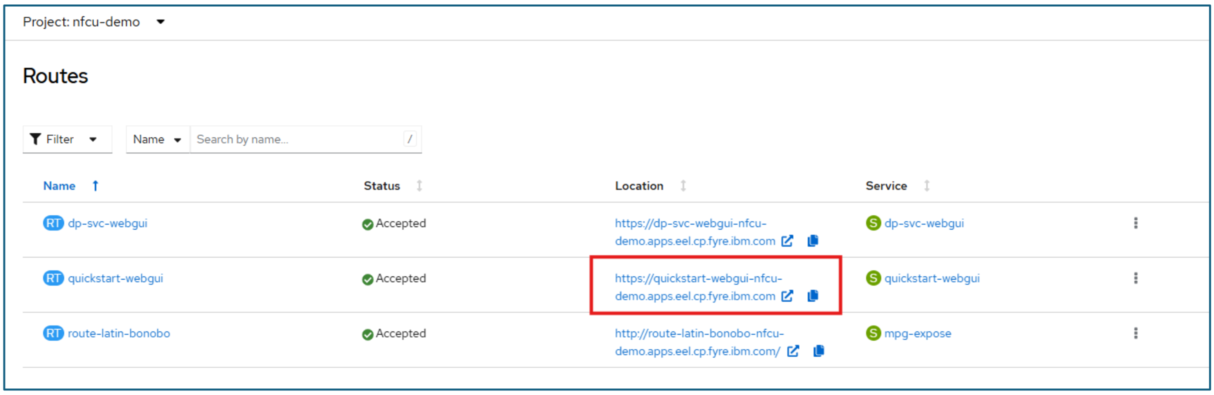
Lab 7: Validate the Deployment

Objective

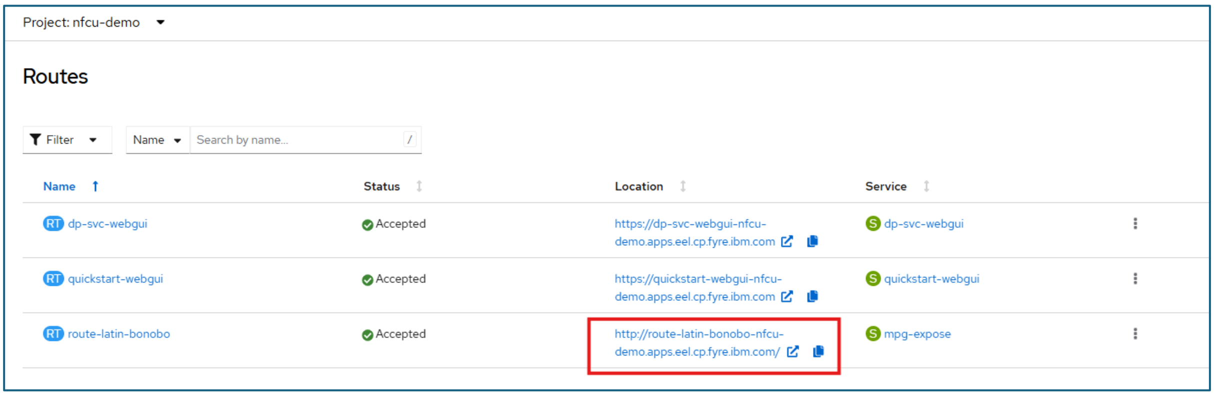
Confirm the deployment works from both the admin and API sides.

Steps

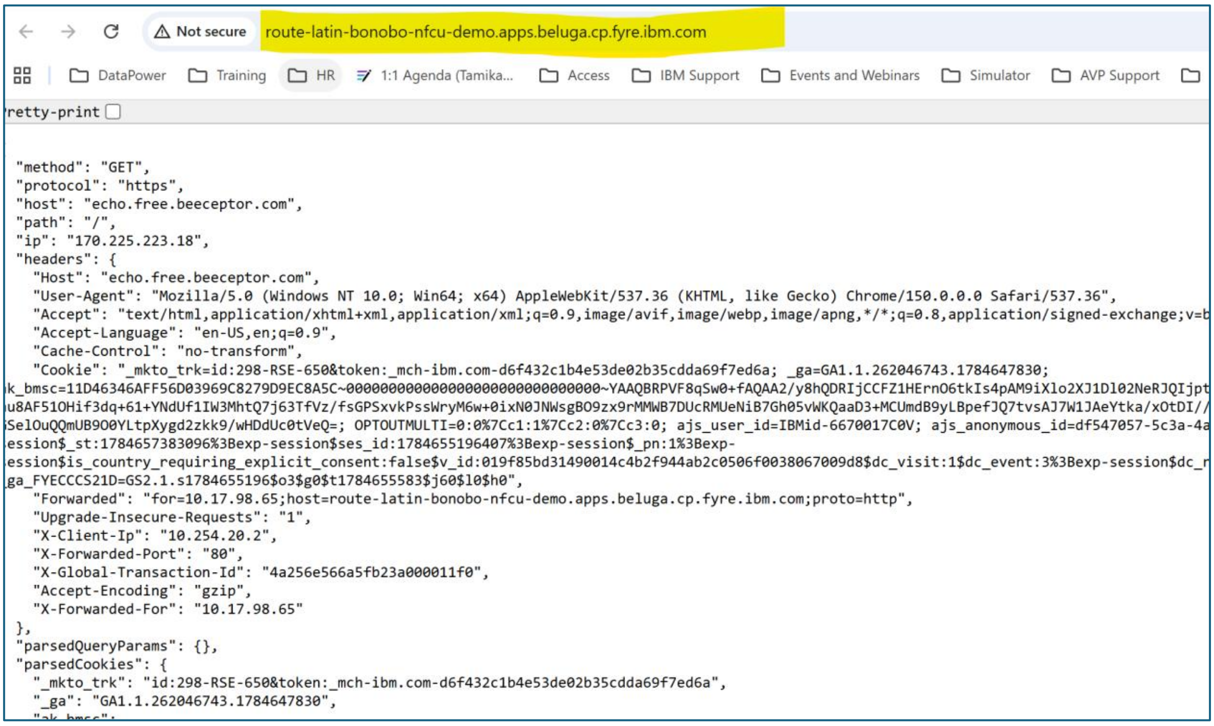
1. Open the web UI in a browser using the `quickstart-webgui` route hostname.
2. **From Console:** OpenShift Console-->Networking--Routes--Select →quickstart-webgui-->Select Location link



2. Log in with:
   - Username: `admin`
   - Password: `admin`
3. Confirm the `default` and `demo` domains are up.
4. Test the MPG route:
   a. Download `mpg-expose.zip` from [Git Hub Repo - Multiprotocol Gateway]()
   b. Import into the `demo` domain.
   c. Test the service
   d. Go to OpenShift Console-->Networking->-Routes→Select Location Link for **route-latin-bonobo.**



e. This is hitting the mpg-expose MPGW defined in the demo domain listening on port 6000. It should return a json object



5. Review cluster objects:

Command: `oc get pods`

Command: `oc get datapowerservice`

Command: `oc get route`

8. Demonstrate the Impact of Missing Persistence

In this step, we will delete the DataPower pod and retest the route to observe that the previously imported service is no longer available. This illustrates what happens when there is **no persistence configured** and no Git repository set up to automatically restore the configuration.

Steps:

a. Delete the DataPower pod:

Command oc delete pod quickstart-0

b. Wait for the pod to restart:

Command oc get pods -w

Wait until quickstart-0 returns to Running status.

c. Retest the MPG route:

- Navigate to **OpenShift Console → Networking → Routes**
- Select the **Location link** for route-latin-bonobo

d. Observe the result:

- The route will **fail** — the service that was previously imported no longer exists
- This is because the DataPower configuration was not persisted to a volume or pulled from a Git repository on startup

Key Takeaway: Without persistence or a GitOps repository, any configuration loaded into DataPower is lost when the pod is deleted or restarted. This demonstrates the importance of the GitOps approach covered in Lab 9.

Checkpoint

- Web UI login works
- Default domain is up
- MPG route responds
- Base platform is ready for the GitOps lab

Lab 8: Pull Configuration from Git

Objective

Add a new DataPower domain that uses the GitOps feature to pull configuration directly from a GitHub repository.

You will create the required secrets and ConfigMap, update the DataPowerService resource, validate the new domain in the DataPower web UI, and confirm that configuration was written back to GitHub.

Steps

Part A: Create Required Resources

1. Make sure you are in the `nfcu-demo` project:

Command: `oc project nfcu-demo`

2. Create the ConfigMap from the `pull-from-git.cfg`.

Command: `oc create configmap pull-from-git-cfg --from-file=pull-from-git.cfg -n nfcu-demo`

```
  Note: This file contains the gitops object from DataPower
```

4. Create the GitHub Personal Access Token (PAT) secret:

(IBM Team Will provide Token Details)

Command: `oc create secret generic pat --from-literal=password=<YOUR_GITHUB_PAT_TOKEN> -n nfcu-demo`

> **Note:** Replace `<YOUR_GITHUB_PAT_TOKEN>` with your GitHub PAT. The PAT must have `repo` scope so DataPower can read from and write to the repository.

5. Verify the ConfigMap exists:

Command: `oc get configmap pull-from-git-cfg -n nfcu-demo`

6. Verify the certificate secret exists:

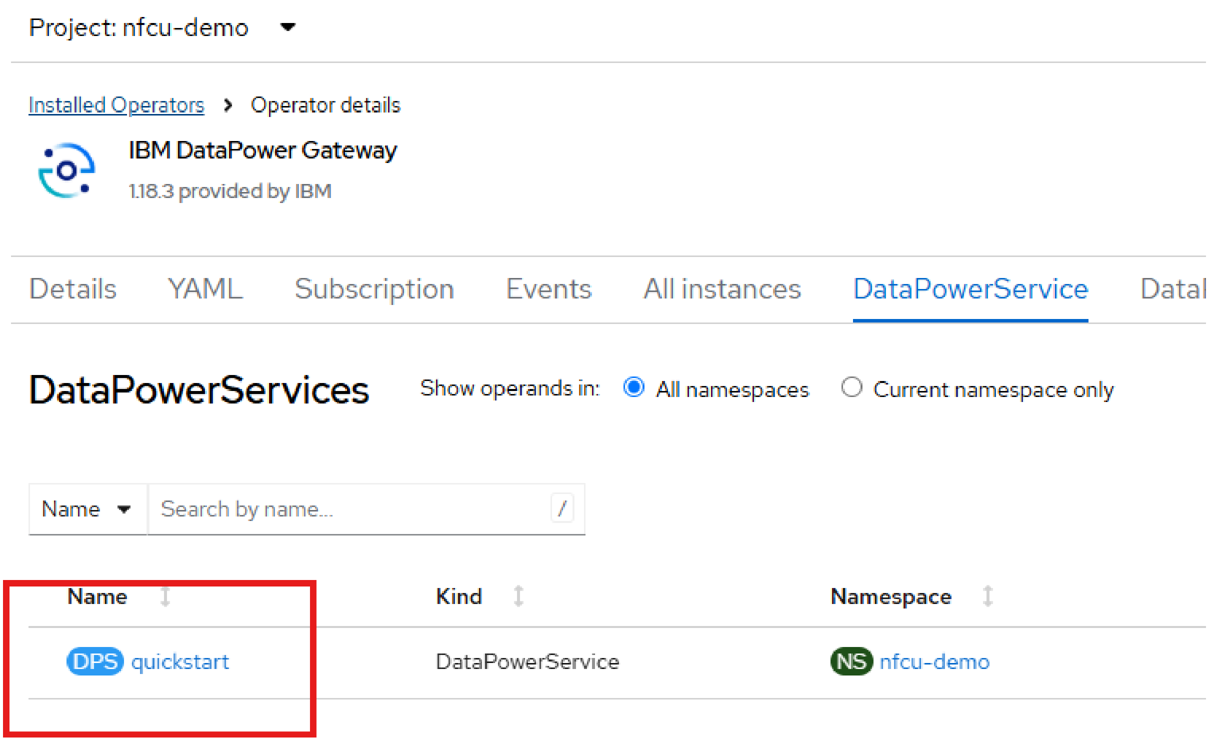
Command: `oc get secret github.com.cer -n nfcu-demo`

7. Verify the PAT secret exists:

Command: `oc get secret pat -n nfcu-demo`

Part B: Update the DataPowerService

8. In the OpenShift console, go to **Operators** > **Installed Operators**.
9. Open **IBM DataPower Gateway**.
10. Click **DataPowerService**.
11. Click the `quickstart` DataPowerService to open it.



12. Select YAML Tab
13. In the far right hand corner select Download.
14. datapowerservice-quickstart.yaml is downloaded to your local file system.
14. Open the datapowerservice-quickstart.yaml in text editor
14. Fine the line the where the domain definitions start – this where we will make an update.

    1. Download the [pot-additionalDomainConfig.yaml]()
    2. Open pot-additionalDomainConfig.yaml
    3. Select All the contents
    4. Paste the into the datapowerservice-quickstart.yaml
    5. You should replace the domains: to name: demo section

Before:

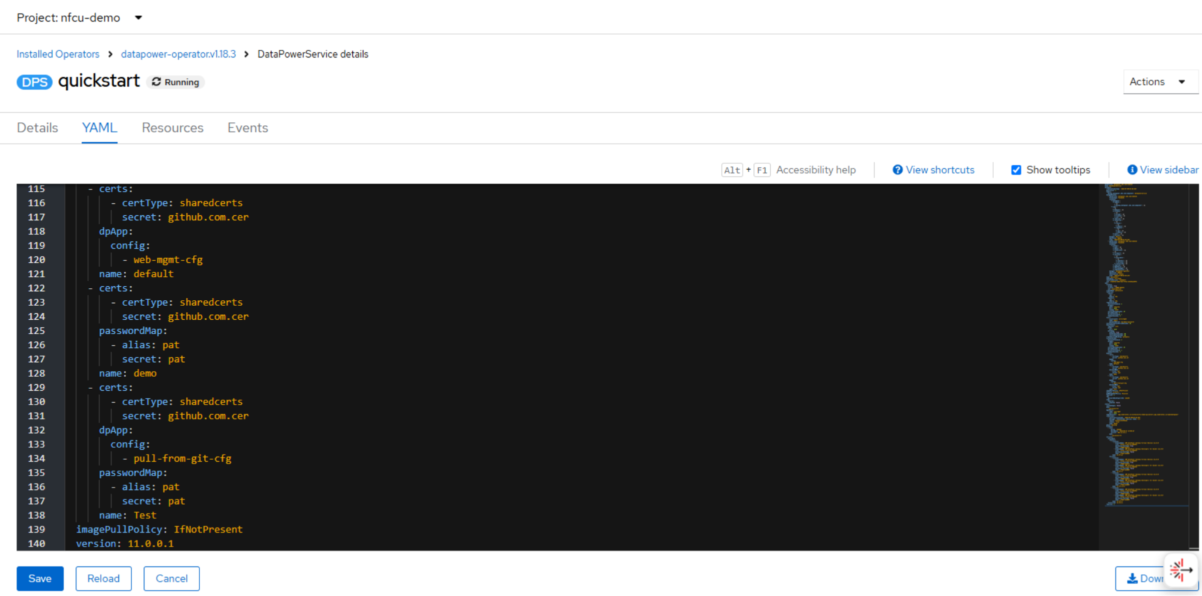
```yaml
domains:
  - certs:
      - certType: sharedcerts
        secret: github.com.cer
    dpApp:
      config:
        - web-mgmt-cfg
    name: default
  - certs:
      - certType: sharedcerts
        secret: github.com.cer
    name: demo
```

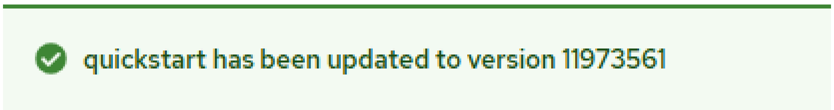
After:
We have added a **config map** to **demo domain** to pull contents from github
We have also added **a new Test domain** to **pull contents from github**

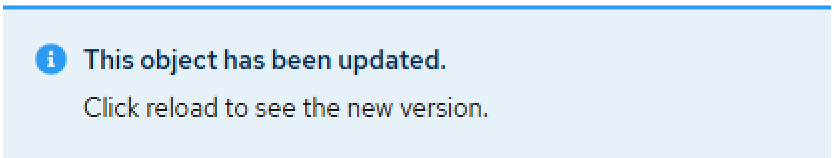
```yaml
domains:
  - certs:
      - certType: sharedcerts
        secret: github.com.cer
    dpApp:
      config:
        - web-mgmt-cfg
    name: default
  - certs:
      - certType: sharedcerts
        secret: github.com.cer
    passwordMap:
      - alias: pat
        secret: pat
    name: demo
  - certs:
      - certType: sharedcerts
        secret: github.com.cer
    dpApp:
      config:
        - pull-from-git-cfg
    passwordMap:
      - alias: pat
        secret: pat
    name: Test
```

15. Save datapowerservice-quickstart.yaml
16. Copy all Contents from the datapowerservice-quickstart.yaml file
17. Go back to quickstart yaml and replace with the contents from updated datapowerservice-quickstart.yaml

18. Click Save







18. Click Reload

Optional: Watch the pod restart and reach `Running`:

Command: `oc get pods -w`

Part C: Validate in the DataPower Web UI

17. **Login to Web UI:**
    - Navigate to **OpenShift Console → Networking → Routes**
    - Select the **Location link** for **quickstart-webgui**

18. Log in with:
    - Username: `admin`
    - Password: `admin`
19. Notice the new **Test** domain listed alongside `default` and `demo`.
20. Switch to the **Test** domain.
21. In the search bar, type `GitOps` and open the **GitOps** object.
    a. Review the GitOps configuration — confirm the repository URL points to: **GitHub Repository URL:** https://github.com/innessrg/NFCU_MVP1_Source_Files.git

    b. Navigate to **Multi-Protocol Gateway** in the **Test** domain and review the new services that were pulled from the repository.

Retest the MPG route:

- Navigate to **OpenShift Console → Networking → Routes**
- Select the **Location link** for route-latin-bonobo

Part D: Confirm Configuration Written Back to GitHub

24. Open the following repository in a browser:

GitHub Repository URL:
https://github.com/innessrg/NFCU_MVP1_Source_Files.git

25. Confirm that the DataPower domain configuration files are present in the repository, confirming that the GitOps write-back completed successfully.

Checkpoint

- `pull-from-git` ConfigMap exists

- `github.com.cer` secret exists
- `pat` secret exists
- DataPowerService updated with `Datapower-NewDomain.yaml`
- Pod restarted and reached `Running`
- **Test** domain appears in the DataPower web UI
- GitOps object is visible and points to the correct repository
- Multi-Protocol Gateway services are visible in the **Test** domain
- DataPower configuration is visible in the GitHub repository

> **Key Takeaway:** Without persistence or a GitOps repository, any configuration loaded into DataPower is lost when the pod is deleted or restarted. The GitOps approach — develop locally with Podman Desktop, Virtual DP, commit to Git, deploy via pipeline — eliminates the need for persistent storage in OpenShift while guaranteeing configuration consistency across every environment.

Looking Ahead — Lab 9: Developing in OpenShift with Persistent Storage

In the previous example, we saw how a GitOps workflow allows you to develop locally and deploy to OpenShift without ever needing persistent storage — Git and a pipeline handle everything.

However, there are scenarios where you may want or need to develop directly within OpenShift — for example, when iterating quickly on a live cluster, troubleshooting in-place, or working in an environment where local tooling is not available.

In Lab 9, we will demonstrate this approach by:

Configuring persistent storage for the DataPower pod in OpenShift

Making configuration changes that survive pod restarts

Exploring how persistent storage can serve as an alternative to a fully automated GitOps pipeline during active development

Note: Persistent storage in this context is not a replacement for GitOps — it is a tool for in-cluster development. Best practice is still to commit finalized configuration.

Troubleshooting: DataPowerService Failed to Create

If the DataPowerService does not come up, check the following:

1. Check the pod status and events:

Command: `oc get pods -n nfcu-demo`

Command: `oc describe pod quickstart-0 -n nfcu-demo`

Look at the **Events** section at the bottom of the `describe` output — it will show pull errors, scheduling failures, or config issues.

2. Check the DataPowerService resource status:

Command: `oc describe datapowerservice quickstart -n nfcu-demo`

Look for any status conditions or error messages under **Status**.

3. Check the operator logs:

Command: `oc logs -n openshift-operators -l name=datapower-operator`

The operator is responsible for creating the pod — its logs will show any reconciliation errors.

4. Check for image pull errors:

If the pod is stuck in `ImagePullBackOff` or `ErrImagePull`, the IBM entitlement secret is likely missing or not linked. Verify with:

Command: `oc get secret ibm-entitlement-key -n nfcu-demo`

Command: `oc get serviceaccount default -n nfcu-demo -o yaml | grep ibm-entitlement-key`

5. Check for missing secrets or ConfigMaps:

If the pod fails with a mount error, a referenced secret or ConfigMap may not exist. Verify all required resources from Lab 4 are present:

Command: `oc get secret,configmap -n nfcu-demo`

6. View events across the namespace:

Command: `oc get events -n nfcu-demo --sort-by='.lastTimestamp'`

This shows a timeline of all recent events and is often the fastest way to identify what went wrong.

Lab 9: Configure Persistent Storage for IBM DataPower on OpenShift

Objective

Configure persistent local storage for IBM DataPower on Red Hat OpenShift so the DataPower pod can use separate persistent mounts for `/config` and `/local` on the same worker node.

Lab Overview

This lab demonstrates how to use the Red Hat Local Storage Operator to provide node-local persistent storage for an IBM DataPower deployment on OpenShift. The design uses two partitions on the same secondary disk to model independent persistent volumes for the two DataPower storage paths listed below.

- `/opt/ibm/datapower/drouter/config`
- `/opt/ibm/datapower/drouter/local`

This separation is useful in a lab because it creates a clear one-to-one relationship between disk partitions, persistent volumes, persistent volume claims, and DataPower mount targets.

Important Design Note

This lab uses **node-local** storage, not shared cluster storage. The persistent volumes created from local partitions are tied to the worker node that owns the disk, so this design preserves data across pod recreation on that node but does not provide cross-node storage failover by itself.

Persistent storage is not a requirement for DataPower containers, nor is it generally best practice. Depends on the use case.. but generally speaking configuration should be managed and mounted via configMaps, secrets and repos, not mutated within the container and persisted to disk.

Any services/configuration that relies on DataPower's RAID capability would utilize persistent storage in k8s to back the file system

Learning Objectives

In this lab, you will:

- Retrieve node details to identify the target deployment node.

- Partition a raw secondary block device (`/dev/vdb`) on Red Hat Enterprise Linux CoreOS.
- Install and configure the Red Hat Local Storage Operator.
- Provision the `local-file-sc` storage class and bind node partitions to local persistent volumes.
- Deploy IBM DataPower with `deleteClaim: false` to preserve persistent state.
- Validate that configuration persists across pod recreation.

Prerequisites

Before starting, ensure the following requirements are met:

- Cluster-admin access to OpenShift is available.
- The `oc` CLI is installed and already logged in to the target cluster.
- A secondary block device is attached to the worker node and available as `/dev/vdb`.

Required DataPower images, entitlement credentials, secrets, and ConfigMaps from earlier labs are already present.

Files referenced in this lab include:

- Datapower.yaml
- Datapower-lso.yaml
- datapower-localvolume.yaml

Reference Documents

Use the following references during the lab:

- Red Hat OpenShift Local Storage Operator documentation.
- Local Storage Operator OLM deployment guidance.
- IBM DataPower Operator storage documentation.
- IBM DataPower initialization and ConfigMap reference documentation.
- IBM architectural guidance for DataPower on OpenShift.

Steps

Exercise 2: Identify the Target Worker Node — Do This First

While quickstart-0 is still running from Lab 8, record the worker node:

oc get pod quickstart-0 -n nfcu-demo -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
quickstart-0	1/1	Running	0	16m	10.0.0.12	worker1.beluga.cp.fyre.ibm.com

Exercise 2: Partition Host Worker Storage

Step 2.1: Locate the Target Node

Because this lab uses local storage, the DataPower pod must run on the worker node that owns the local partitions and persistent volumes.

For Example:
worker1.beluga.cp.fyre.ibm.com

Step 2.2: Access the Host Shell

Start a debug session on the target worker node.

oc debug node/[UPDATE NODE FROM COMMAND IN STEP 2.1]
for example
oc debug node/**worker1.beluga.cp.fyre.ibm.com**

```
a.cp.fyre.ibm.com
Temporary namespace openshift-debug-gcpzh is created for debugging node...
Starting pod/worker1belugacpfyreibmcom-debug-mz9kw ...
To use host binaries, run `chroot /host`
Pod IP: 10.17.123.179
If you don't see a command prompt, try pressing enter.
sh-5.1#
```

Switch to the host filesystem.

```
chroot /host
```

Step 2.3: Partition /dev/vdb

Partition the disk using `fdisk`.

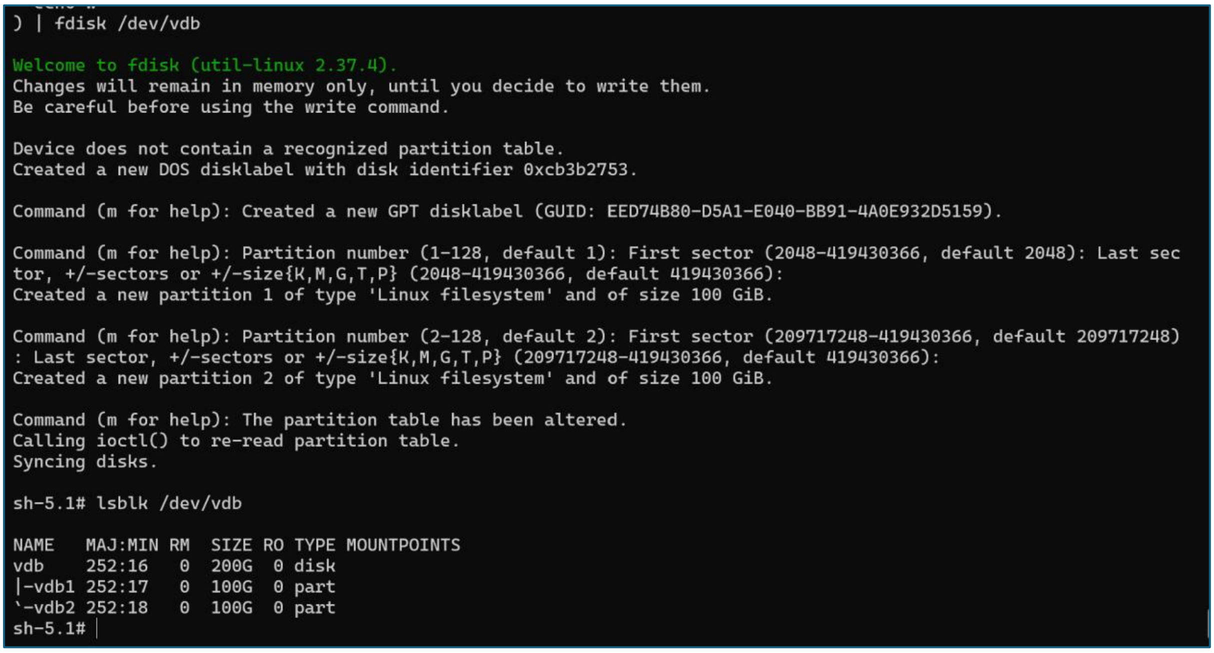
Copy and Paste the Following text into the debug session:

```
(
 echo g
 echo n
 echo 1
 echo
 echo +100G
 echo n
 echo 2
 echo
 echo
 echo w
) | fdisk /dev/vdb
```

Verify the partitions were created.

```
lsblk /dev/vdb
```

Expected result: `/dev/vdb1` and `/dev/vdb2` exist and are available for local PV creation.



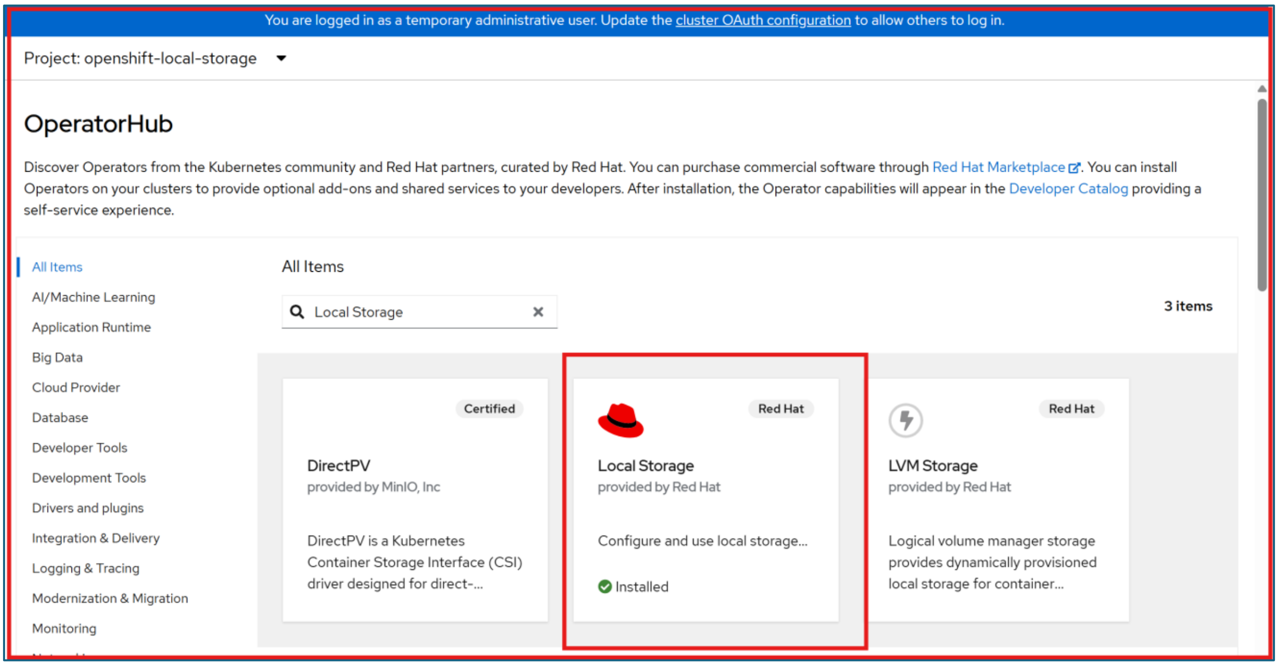
```
) | fdisk /dev/vdb

Welcome to fdisk (util-linux 2.37.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0xcb3b2753.

Command (m for help): Created a new GPT disklabel (GUID: EED74B80-D5A1-E040-BB91-4A0E932D5159).

Command (m for help): Partition number (1-128, default 1): First sector (2048-419430366, default 2048): Last sec
tor, +/-sectors or +/-size{K,M,G,T,P} (2048-419430366, default 419430366):
Created a new partition 1 of type 'Linux filesystem' and of size 100 GiB.

Command (m for help): Partition number (2-128, default 2): First sector (209717248-419430366, default 209717248)
: Last sector, +/-sectors or +/-size{K,M,G,T,P} (209717248-419430366, default 419430366):
Created a new partition 2 of type 'Linux filesystem' and of size 100 GiB.

Command (m for help): The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

sh-5.1# lsblk /dev/vdb

NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vdb    252:16   0  200G  0 disk
|-vdb1 252:17   0  100G  0 part
`-vdb2 252:18   0  100G  0 part
sh-5.1# |
```

Exit the debug session.

```
exit
exit
```

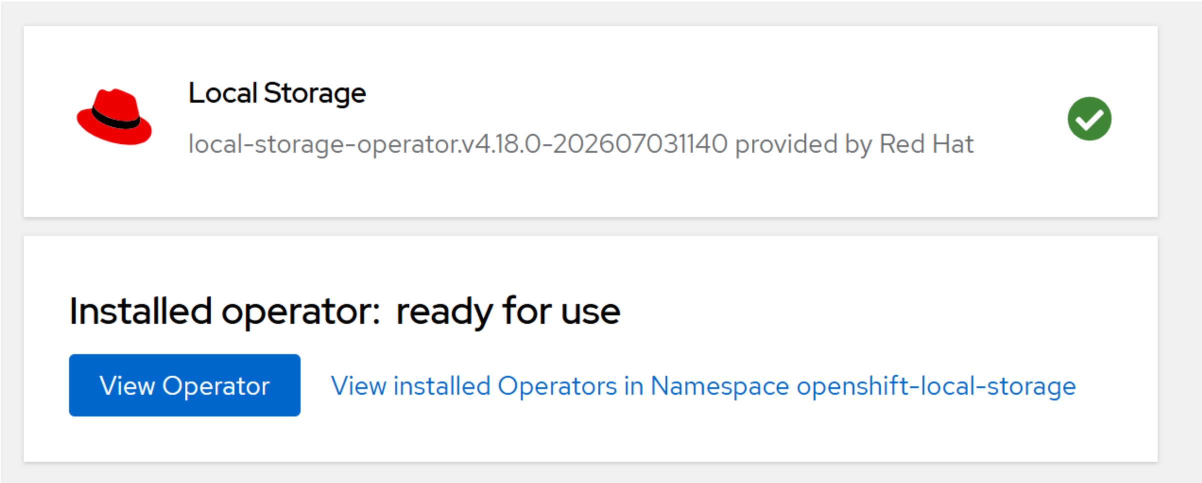
Exercise 3: Install the Local Storage Operator

In the OpenShift web console, go to **Operators** > **OperatorHub** and search for **Local Storage**. Select the **Red Hat Local Storage Operator** tile and install it with the following settings.



- Update Channel: `stable`
- Installation Mode: A specific namespace on the cluster
- Installed Namespace: `openshift-local-storage`
- Update Approval: `Automatic`

Wait until the operator status shows **Succeeded** under **Operators** > **Installed Operators**.



Exercise 4: Provision the StorageClass and Persistent Volumes

Step 4.1: Review datapower-localvolume.yaml file

Review datapower-localvolume.yaml from the repository before applying it. This manifest should define the local storage resources that discover and expose the partitions as local persistent volumes.

Step 4.2: Update datapower-localvolume.yaml with the correct node. (Recorded from Exercise 1)

```
nodeSelector:
  nodeSelectorTerms:
    - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
            - <your-actual-worker-node>   # ← update this line
```

For Example:

```yaml
apiVersion: local.storage.openshift.io/v1
kind: LocalVolume
metadata:
  name: datapower-local-volumes
  namespace: openshift-local-storage
spec:
  nodeSelector:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - worker1.beluga.cp.fyre.ibm.com   # ← update this line
  storageClassDevices:
    - storageClassName: local-file-sc
      volumeMode: Filesystem
      fsType: ext4
      devicePaths:
        - /dev/vdb1
        - /dev/vdb2
```

Step 4.3: Save File

Apply the LocalVolume configuration.

```
oc apply -f datapower-localvolume.yaml
```

Verify that the storage class exists.

```
oc get sc local-file-sc
```

Verify the local persistent volumes.

```
oc get pv | grep local-file-sc
```

Ensure both entries are associated with `local-file-sc` and show as `Available` before moving forward.

Exercise 5: Clean the Cluster and Redeploy DataPower with Local Storage

Step 5.1: Review `Datapower-lso.yaml` carefully before applying it.

Confirm that the manifest contains two storage definitions, that the storage is mapped to the correct DataPower mount paths, that the storage class references are correct, and that `deleteClaim: false` is configured on the persistent storage entries.

Step 5.2 Delete the previous DataPower Service that contained the github configuration.

Apply the new DataPower Service that references the Local Storage.

```
oc delete datapowerservice quickstart -n nfcu-demo --ignore-not-found
oc apply -f Datapower-lso.yaml -n nfcu-demo
```

Exercise 6: Validate and Test Persistence

Step 6.1: Verify Pod and PVC Binding

Verify the persistent volume claims are bound.

```
oc get pvc -n nfcu-demo
```

Note: oc get pvc -n nfcu-demo shows that both PersistentVolumeClaims required by the DataPower instance have been successfully created and bound to PersistentVolumes. The claims quickstart-datapower-drouter-config-quickstart-0 and quickstart-datapower-drouter-local-quickstart-0 are in Bound status, confirming that storage has been allocated correctly using the local-file-sc storage class.

Verify the DataPower pod is running.

```
oc get pods -n nfcu-demo
```

Step 6.2: Write and Commit Test Configurations

Open the DataPower Web UI in your browser.

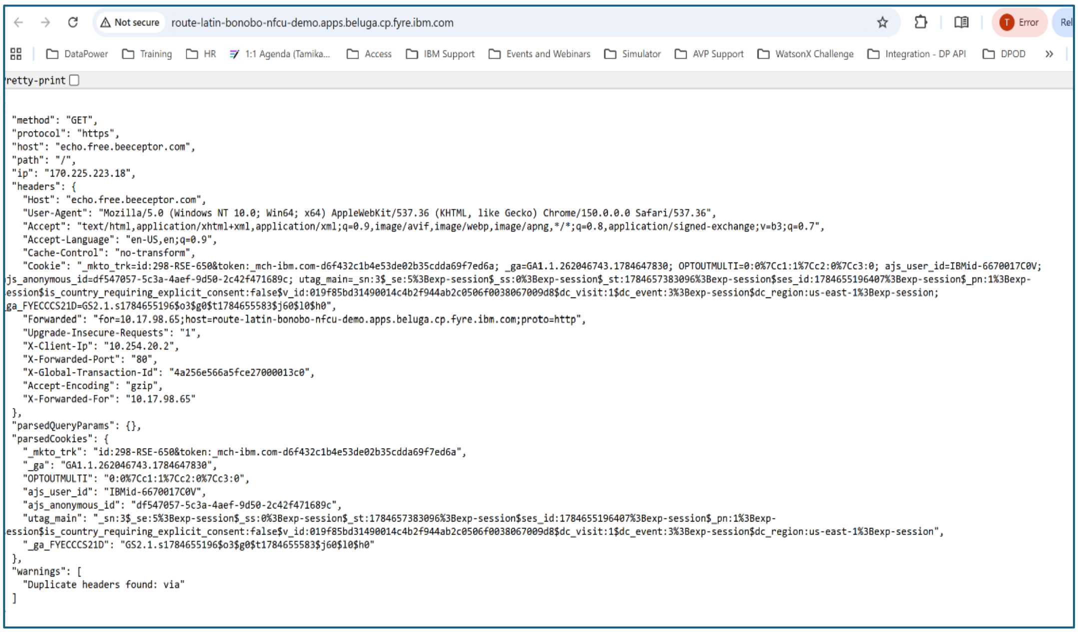
Login to Web UI:

- Navigate to **OpenShift Console → Networking → Routes**
- Select the **Location link** for **quickstart-webgui**
- Go to the demo domain

- Import your configuration assets, for example `mpg-expose.zip`,
- Then click **Save Configuration** in the top-right corner of the WebGUI to commit changes to disk.

Retest the MPG route:

- Navigate to **OpenShift Console → Networking → Routes**
- Select the **Location link** for route-latin-bonobo
- Verify it returns Valid json



Step 6.3: Run the Pod Recreation Test

Delete the running DataPower pod.

```
oc delete pod quickstart-0 -n nfcu-demo
```

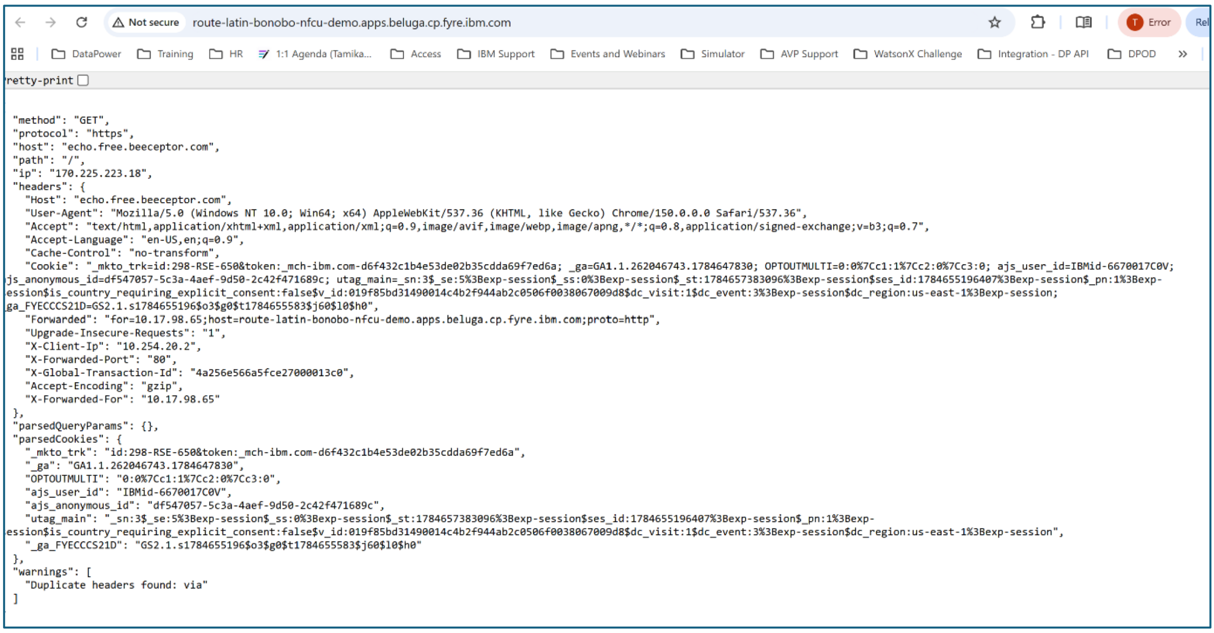
Watch the replacement pod come back.

```
oc get pods -n nfcu-demo -w
```

After the new `quickstart-0` pod is running, log back in and confirm that the imported configuration is still present.

Retest the MPG route:

- Navigate to **OpenShift Console → Networking → Routes**
- Select the **Location link** for route-latin-bonobo
- Verify it returns Valid Json



Checkpoint

At this stage, confirm the following:

- Existing DataPower resources and stale storage mappings were cleaned up.
- The target worker node was identified.
- `/dev/vdb1` and `/dev/vdb2` were created successfully.
- The Local Storage Operator is installed and running.
- `local-file-sc` exists.
- Local persistent volumes are available.
- DataPower deployed successfully with `deleteClaim: false`.
- PVCs are bound.
- The WebGUI is reachable.
- Imported configuration persists after pod recreation.

Troubleshooting: DataPowerService Failed to Create

If the DataPowerService does not come up, use the following checks.

1. Check pod status and events

```
oc get pods -n nfcu-demo
oc describe pod quickstart-0 -n nfcu-demo
```

Review the `Events` section at the bottom of the describe output for image pull errors, scheduling failures, and configuration problems.

2. Check the DataPowerService resource status

```
oc describe datapowerservice quickstart -n nfcu-demo
```

Review any status conditions or error messages under `Status`.

3. Check the operator logs

```
oc logs -n openshift-operators -l name=datapower-operator
```

The operator is responsible for creating the pod, so reconciliation failures often appear here first.

4. Check for image pull errors

If the pod is stuck in `ImagePullBackOff` or `ErrImagePull`, verify that the IBM entitlement secret exists and is linked correctly.

```
oc get secret ibm-entitlement-key -n nfcu-demo
oc get serviceaccount default -n nfcu-demo -o yaml | grep ibm-entitlement-key
```

5. Check for missing secrets or ConfigMaps

If the pod fails with a mount error, verify that all referenced secrets and ConfigMaps from earlier labs are present.

```
oc get secret,configmap -n nfcu-demo
```

6. Review namespace events

```
oc get events -n nfcu-demo --sort-by='.lastTimestamp'
```

This command provides a useful event timeline and is often the quickest way to identify the immediate cause of a failed deployment.

Lab Notes for Production Design

This lab is intentionally designed for visibility and learning rather than for a production-ready storage architecture. In production, teams often use resilient shared storage or platform-native block storage instead of node-local partitions, but the separation of persistent DataPower storage concerns remains a useful design principle.